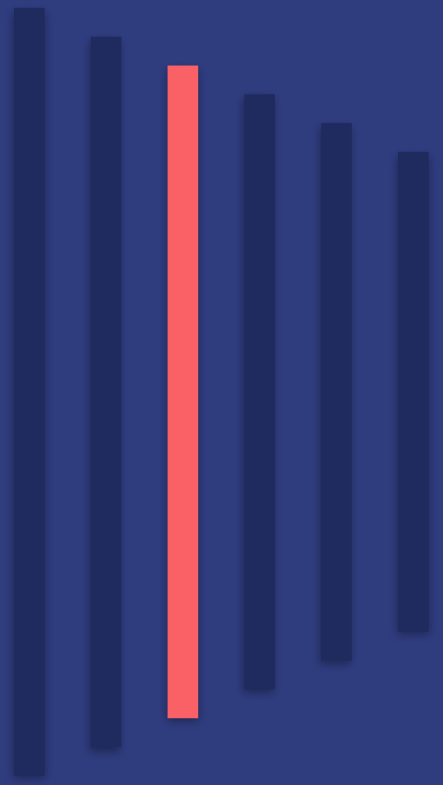
LESSON 04

Skill 设计与能力沉淀

造接口给 AI 用

SKILL.md | Skill 市场 | backtest | brainstorming



25 min · 造接口给 AI 用

AGENDA · 今日路线图

四件事，递进展开

01 Skill 是什么

Prompt 一次性，
Skill 可复用

五种扩展方式对比 · Skill 文件结构

02 Skill 市场速览

别从零造，
先看现成

官方 / Superpowers / ECC / skill.sh / skillsmp

03 写 backtest skill

30 行 Python +
30 行 Markdown

backtest.py · SKILL.md · example_strategies

04 用 skill 跑回测 + 作业

看真实对比 +
设计作业

真实数字 · 反模式 · 提交清单

01

Skill 是什么

Prompt 一次性，Skill 可复用

01 · 五种扩展方式对比

Prompt / Rule / Subagent / MCP / Skill—— 各有定位

	Prompt	Rule	Subagent	MCP	Skill
触发	你打字	自动 / glob	你主动派遣	AI 判断调用	AI 匹配触发
上下文	主线对话	主线注入	独立子上下文	主线注入工具	主线对话
谁定义	你每次写	你 / 团队	你 / 官方	第三方 / 你	你 / 社区
例子	重命名变量	UTC 约定	探索代码库	sqlite 查询	backtest

description 是 Skill 的招牌

SKILL.md — frontmatter

```
---
name: backtest
description: Use when the user asks
  to backtest a crypto trading
  strategy — translate to callable
  → call run_backtest → explain.
---
```

sections （markdown body）

何时触发

触发条件——告诉 Claude 何时自动调用

步骤

3–7 条原子化步骤，每步一句话

反模式

明确不该做什么，约束 Claude 行为

示例

输入 / 输出示例，可链接到示例文件

description 写模糊 → Claude 乱触发或不触发；写具体 → 触发精准

02

Skill 市场速览

别从零造，先看现成

5 个来源，各有侧重

Anthropic 官方 skills github.com/anthropics/skills	reference skill 集合，最权威
Superpowers github.com/obra/superpowers	方法论框架（lesson 2 已装），含 brainstorming / TDD / code-review
Everything Claude Code (ECC) github.com/affaan-m/everything-claude-code	Claude Code 增强包，含工程类 skill （build / lint / review）
skill.sh skill.sh	社区聚合站，按主题分类，支持搜索安装
skillsmp skillsmp.com	另一个社区聚合站，规模较小，垂直领域丰富

4 条选择心法

1 找标准参考实现

官方仓库（github.com/anthropics/skills）

最权威，适合初学理解 SKILL.md 规范

2 找工作流 skill

Superpowers （github.com/obra/superpowers）

brainstorming / TDD / code-review / systematic-debugging ， 已在 lesson 2 装过

3 找工程类 skill

Everything Claude Code (ECC)

build / lint / review / test-coverage 等工程辅助，集中在一个 plugin 里

4 找冷门垂类

skill.sh / skillsmp 搜

量化 / 运维 / 生信 / 医疗等垂直领域，社区贡献，质量不一需甄别

03

写一个 backtest skill

30 行 Python + 30 行 Markdown

03 · backtest 设计目标

输入策略 callable → 输出 4 项指标 + PNG + 中文总结

输入

strategy callable: def strategy(df) -> Series

symbol: 默认 BTCUSDT

interval: 默认 1d

limit: K 线行数，默认 30

输出

sharpe: 夏普比（年化）

max_drawdown: 最大回撤（如 -0.0424）

win_rate: 信号触发日胜率

cumulative_return: 累计收益率

PNG: cum_BTCUSDT.png 曲线图

summary: ~120 字中文总结段

03 · backtest.py 关键设计

策略 callable + signal.shift(1) 防 lookahead

lesson4/project/backtest.py （核心片段）

```
def run_backtest(strategy, symbol="BTCUSDT",
                 interval="1d", limit=30):
    df = fetch_ohlc(symbol, interval, limit)   # 从 prices.db 取数
    sig = strategy(df)                          # 策略 callable
    # ★ shift(1): 今天信号基于昨天价格，防 lookahead bias
    pos = sig.shift(1).fillna(0)
    ret = df["close"].pct_change().fillna(0)
    strat_ret = pos * ret
    sharpe  = strat_ret.mean() / strat_ret.std() * (252**0.5)
    cum_ret = (1 + strat_ret).cumprod() - 1
    max_dd  = (cum_ret - cum_ret.cummax()).min()
    win_rate = (strat_ret > 0).sum() / (pos != 0).sum()
    # 画曲线 → cum_BTCUSDT.png
    # 生成中文 summary

    return {"sharpe": sharpe, "max_drawdown": max_dd, ...}
```

★ *signal.shift(1) 是防 lookahead 的核心——今天收盘决定明天仓位，不允许用未来数据*

description 写好写坏，差距在哪里

坏例子

```
---
name: backtest
description: 回测工具
---
```

- ✗ 太宽泛——Claude 不知何时触发
- ✗ 没有说输入是什么
- ✗ 没有说输出是什么
- ✗ 容易误触发 / 不触发

好例子

```
---
name: backtest
description: Use when user asks to
  backtest a trading strategy —
  translate to callable → call
  run_backtest → explain result.
---
```

- ✓ 说清楚「何时用」（用户要求回测）
- ✓ 说清楚「做什么」（翻译 → 调 → 解读）

03 · SKILL.md 步骤段

5 步——把流程教给 Claude 一次，永久复用

1 翻译策略

把用户自然语言描述写成 Python callable

def strategy(df) -> Series，值为 +1/0/-1

2 校验

dry-run：信号边界对吗？有无 lookahead？

（shift 由 run_backtest 内部处理，无需手写）

3 调 run_backtest

from backtest import run_backtest

result = run_backtest(strategy, ...)

4 解读结果

用 result['summary'] 给文字总结；

告诉用户 PNG 路径在 result['png_path']

5 可选对比

用户要同时对比多个策略时，重复 1-4 各一次，最后写对比段

03 · example_strategies.py

两个起手模板—— Claude 翻译策略时的参考

buy_and_hold · 基线

```
def buy_and_hold(df):
    """始终满仓做多，基线对照"""
    return pd.Series(
        1, index=df.index
    )
```

始终持有，信号恒为 +1 。
用途：给任何策略提供基准对照——
跑赢它才有意义。

ma_crossover_7 · 均线择时

```
def ma_crossover_7(df):
    """close > 7日均线 → 做多"""
    ma7 = df["close"].rolling(7).mean()
    sig = (df["close"] > ma7)
    return sig.map({True:1, False:0})\
              .fillna(0)
```

收盘价 >7 日均线时做多，否则空仓。
最简趋势跟踪—— Claude 翻译用户策略
时参考信号格式和 fillna 写法。

04

用 skill 跑回测

看真实对比 + 作业

两个策略对比——真实数字（2026-05-05 行情）

Prompt （整段粘贴）

用 backtest skill 评估： 1. buy_and_hold 2. ma_crossover_7 各 30 天日 K 线（BTCUSDT），给我对比报告。

Skill 回测输出（真实数字）

```
Skill(backtest) — strategy 1: buy_and_hold
  { sharpe: 4.6545,  max_drawdown: -0.0424,
    win_rate: 0.6552,  cumulative_return: 0.1737 }

Skill(backtest) — strategy 2: ma_crossover_7
  { sharpe: 2.2799,  max_drawdown: -0.0424,
    win_rate: 0.6471,  cumulative_return: 0.0451 }
```

刚才发生了什么——三件值得注意的事

1

Claude 自己翻译策略为 callable

你没写任何 Python——Claude 看了 SKILL.md 里的约定和 example_strategies.py 的写法，自己生成了正确的策略函数。

2

主线只看到两次 skill 调用

整个对话里只有两次 Skill(backtest) 被调用，翻译 / 执行 / 解读全部发生在 Claude 内部，不需要你干预。

3

复用而非重写

下次你要测新策略， prompt 里说「用 backtest skill 评估……」即可——不需要重新解释「回测是什么」， Skill 已经记住了流程。

MCP 消费接口 vs Skill 造接口给 Claude

MCP （Lesson 3）

- AI 消费 外部系统的接口
- sqlite MCP → Claude 执行 SQL
- Claude 判断何时调用 Tool
- 外部系统提供能力
- 适合：查数据、调 API 、读文件

VS

Skill （Lesson 4）

- 你 生产 接口给 Claude 用
- backtest Skill → Claude 按流程回测
- Claude 看到匹配意图就触发
- 你定义能力和流程
- 适合：高频 + 流程固定的任务

MCP 负责连外部系统； Skill 负责封装可复用流程。两者互补，不冲突。

学会之前先别踩坑

AP1

写 500 行手册式 SKILL.md

把所有可能情况都写进 SKILL.md，
Claude 读到一半就搞混，
触发时犹豫不定。

改：步骤保持 3-7 条，每条一句话；
细节放 Python 代码里。

AP2

让 Skill 决定业务规则

把「夏普 > 2 才算好策略」
硬写进 SKILL.md 。

改：业务判断标准放 Rule 或 prompt；
Skill 只管流程，不管业务评判。

AP3

SKILL.md 内嵌密钥 / 私有路径

把 API key 或内网路径
写进 SKILL.md，会被 commit
进仓库并被 Claude 每次读到。

改：密钥用 os.getenv()，
SKILL.md 只写变量名。

04 · 作业 Task 1 概览

从日常工作挑一个高频任务设计成 SKILL.md

A

代码审查

输入： git diff

输出：风险列表 + P0/P1/P2 等级

B

日志分析

输入： stderr/log 片段

输出：根因推断 + 修复建议

C

接口文档

输入：函数签名 / OpenAPI spec

输出：中文说明 + 请求示例

D

SQL 审查

输入： SQL 语句

输出：风险点 + 性能提示 + 改写建议

E

commit message

输入： git diff / 改动列表

输出：规范 commit message （Conventional Commits）

选一个你工作里实际用得到的。不要求实现底层代码——只需要 SKILL.md 。

04 · 作业提交清单

30 分钟内可完成，不需要写代码

评分维度（3 条）

1 description 精准

Claude 看了就知道何时触发、输入输出是什么

2 步骤原子化

每步一个动作，3-7 条，Claude 不需要猜

3 反模式明确

至少写出 2 条「不要做什么」约束

提交清单

✓ **SKILL.md**

按 frontmatter + 步骤段 + 反模式格式写出完整 SKILL.md

✓ **3 句话设计说明**

用途 / 触发条件 / 输入输出，各一句

✓ **加分项截图（可选）**

实际在 Claude Code 里跑一次，提交对话截图

提交方式：SKILL.md + 3 句话说明（+ 可选截图），发到课程群即可。

学完这课你能做什么

4 条 Takeaway

1 用 SKILL.md 沉淀高频任务

把一件事教给 Claude 一次，之后按需触发，
不再每次重新解释流程。

2 区分 5 种扩展方式

Prompt / Rule / Subagent / MCP / Skill，
知道什么场景用什么，不瞎用。

3 逛 5 个 Skill 市场

官方 / Superpowers / ECC / skill.sh / skillsmp，
不从零造轮子，先搜现成。

4 description 让 Claude 准确触发

避免「写了 Skill 但 Claude 从不用它」的问题；
写清楚输入、输出、触发时机。

下节课（Lesson 5）：用 OpenSpec 给 backtest 加新策略（双均线）——走完 proposal → specs → tasks → implement 全流程

预告 · Lesson 5

用 OpenSpec 给 backtest 加新策略

任务目标

把「双均线策略（MA7 穿越 MA14）」作为 feature，
用 OpenSpec 走完 proposal → specs → design → tasks → implement 全流程。

本节课写好的 backtest Skill 在 Lesson 5 里会继续被消费——造好接口的价值在后续 lesson 里不断兑现。

Proposal	→	Specs	→	Design	→	Tasks	→	Implement
提出 feature，说明动机		OpenSpec 形式化需求描述		接口设计 + 数据结构		拆分可执行开发任务		Claude 逐一实现任务

你会看到：有了 backtest Skill，Lesson 5 的 implement 阶段只需要描述策略，Claude 自己接好引擎。

Have fun · 延伸阅读

Skill 生态还在扩展——造好接口，社区共享

Anthropic 官方 skills

`github.com/anthropics/skills`

reference skill 集合， SKILL.md 规范最权威，适合入门参考

Superpowers （已在 lesson 2 装过）

`github.com/obra/superpowers`

brainstorming / TDD / code-review 等方法论 Skill，开箱即用

Everything Claude Code (ECC)

`github.com/affaan-m/everything-claude-code`

工程类 skill 集合（build / lint / test / review），含本课 skill marketplace

Anthropic Blog — Agent Skills

`anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills`

官方关于 Skill 设计的最佳实践博客，讲清楚 description 的重要性